

# Events

Query and transaction events as a JSON stream.



## Install & load

```
INSTALL events FROM community;
LOAD events;
```

## 01 At a glance

Forward DuckDB lifecycle events to an executable handler as JSON lines on standard input. Use the stream for operational logs, query timing and transaction traces, with a session tag to identify each running process.



## 02 Four settings & event keys

**events\_destination**

Absolute path to the executable that consumes JSON lines.

**events\_types**

List of enabled query, connection or transaction event types.

**events\_session\_name**

Tag each event with a process or workload identifier.

**events\_async**

TRUE enables asynchronous delivery; default is FALSE.

**query\_id / transaction\_id**

Correlate query pairs and the enclosing transaction.

**has\_error / error\_message**

Inspect failures on query\_end and transaction\_rollback events.

## 03 Keep in mind

### A handler is required

Create an executable with a shebang at the configured absolute path. It must read stdin line by line and persist or process each JSON object. The extension launches the handler with DuckDB's privileges.

## 04 Capture query activity

### MANUAL SETUP

```
SET events_session_name = 'analytics';
SET events_types = [
  'query_begin', 'query_end'
];
SET events_async = TRUE;
SET events_destination =
  '/usr/local/bin/query-events';
SELECT sum(i) FROM range(1000) t(i);
```

Requires an installed compatible Events build and your executable JSONL handler.

## 05 Trace transaction boundaries

### MANUAL SETUP

```
SET events_types = [
  'transaction_begin',
  'transaction_commit',
  'transaction_rollback'
];
BEGIN;
SELECT 42;
COMMIT;
```

With the handler running, group transaction events by transaction\_id.

## 06 In practice

### Lifecycle and availability

Settings belong to the running process; reapply them at startup. Async delivery trades reliability for query latency. Persist events in the handler when durability matters. Community package availability varies by DuckDB version and platform.

[Full docs & examples ↗](#) [Source ↗](#)